Universidad Icesi

Arquitectura de APIs RESTful en Spring Boot

Sesión 1: Recursos, Versionamiento, Idempotencia, Caché, DTOs, Streams y MapStruct

Computación en Internet II — Universidad Icesi

Ruta de Aprendizaje — 4 Micro-Ciclos (Teoría ↔ Práctica)

Primera Mitad: Fundamentos REST y Contrato DTO

01. REST Orientado a Recursos, Caché e Idempotencia

URIs, Versionamiento, Verbos HTTP, Caché 304, Status Codes + Diagnóstico de antipatrones REST.

02. El Patrón DTO (Data Transfer Object)

Riesgos de exponer `@Entity` + Implementación de contratos asimétricos Request/Response.

Enfoque Práctico: Cada bloque teórico va seguido inmediatamente de su implementación en vivo sobre el dominio de `Usuario`.

Segunda Mitad: Streams, MapStruct y @RestController

03. Java Streams API y Mapeo con MapStruct

Pipelines `.stream().filter().map().collect()`, JSR 269 + Implementación de `Mapper`.

04. @RestController, Parámetros y ResponseEntity

`@PathVariable`, `@RequestParam`, `@RequestBody`, `ResponseEntity` + Controlador REST.

Convención de Rutas: Todos los recursos RESTful siguen el esquema jerárquico versionado `/api/v1/{recurso}` (ej. `/api/v1/usuarios`).

Universidad ICESI

01. REST Orientado a Recursos, Versionamiento, Idempotencia y Caché

Evolución de MVC a REST, diseño de URIs, semántica HTTP (RFC 9110), caché y códigos de estado.

Universidad ICESI

Evolución Arquitectónica: @Controller (MVC) vs. @RestController

Spring MVC Tradicional (@Controller)

Orientado a **Server-Side Rendering (SSR)**. El controlador puebla un objeto `Model` y retorna el nombre de una plantilla HTML.

Motor ViewResolver + Thymeleaf

Resuelve `"usuarios/lista"` hacia un archivo `.html` y entrega `Content-Type: text/html`.

Limitación Multi-Cliente

Una app móvil nativa iOS/Android o un otro microservicio no puede consumir vistas HTML acopladas.

API RESTful Desacoplada (@RestController)

Combina `@Controller + @ResponseBody`. El controlador retorna datos puros (DTOs) que se escriben directamente en el cuerpo HTTP.

Motor HttpMessageConverter (Jackson)

Serializa objetos Java a JSON (`application/json`) sin pasar por ningún motor de vistas.

Interoperabilidad Universal

Un único backend alimenta SPAs (React/Angular), apps móviles, Postman y sistemas externos.

Universidad ICESI

Flujo Interno en Spring: ViewResolver vs. HttpMessageConverter

Procesamiento del Retorno en DispatcherServlet

Comparación del pipeline de salida entre un controlador de vistas MVC y un controlador RESTful

ARQUITECTURA MVC

1. @Controller (Vistas HTML)

El método retorna un **String (Nombre de Vista)**

```
@Controller
public String listar(Model model) {
model.addAttribute("usuarios", list);
return "usuarios/lista";
}
```

COMPONENTE DE SALIDA

ThymeleafViewResolver → HTML
Content-Type: text/html;charset=UTF-8

Estado de sesión: **Stateful (Cookie JSESSIONID)**

ARQUITECTURA REST

2. @RestController (Datos JSON)

El método retorna **ResponseEntity<DTO>**

```
@RestController // @Controller + @ResponseBody
public ResponseEntity<List<UsuarioDTO>> get() {
List<UsuarioDTO> dtos = mapper.toDTOs(list);
return ResponseEntity.ok(dtos);
}
```

COMPONENTE DE SALIDA

MappingJackson2HttpMessageConverter
Content-Type: application/json + HttpStatus

Estado de sesión: **Stateless (Token JWT Bearer)**

REST vs. RESTful: Los 4 Pilares Arquitectónicos (Roy Fielding)

1. RECURSOS

Orientado a Recursos
Toda entidad del dominio (`Usuario` , `Curso`) es un *recurso* identificado por una URI única:
`/api/v1/usuarios/5`
Las URIs nombran sustantivos, jamás verbos.

2. INTERFAZ UNIFORME

Verbos HTTP Estándar
La operación que se desea ejecutar se expresa únicamente mediante el método HTTP:
`GET, POST, PUT, PATCH, DELETE` sobre la misma URI.

3. REPRESENTACIONES

Intercambio de Estado
El cliente nunca accede a la tabla SQL; intercambia *representaciones* serializadas en `application/json` modeladas mediante objetos **DTO**.

4. STATELESS & CACHE

Sin Estado y Cacheable
Cada petición contiene todo el contexto necesario para procesarse, y las respuestas de lectura declaran si pueden almacenarse en **caché**.

Diseño REST Orientado a Recursos: Sustantivos vs. Acciones RPC

Antipatrón: Diseño Orientado a Acciones (RPC)

Acopla el nombre de la función Java en la URL e ignora la semántica del verbo HTTP (típico error al migrar de MVC a REST):

GET /api/obtenerTodosLosUsuarios
Incorrecto: acopla un verbo de acción en la ruta y usa *camelCase* arbitrario.

GET /api/crearUsuario?nombre=Ana
Violación grave de RFC 9110: utiliza un verbo de lectura (`GET`) para insertar datos en la BD.

POST /api/borrarUsuarioPorId?id=4
Incorrecto: usa `POST` en lugar de `DELETE` y esconde la identidad en un query param.

Estándar RESTful: Sustantivos en Plural y Jerarquía

Las URIs representan **colecciones** e **instancias** en minúsculas (*kebab-case*), delegando la acción al método HTTP:

Colección: /api/v1/usuarios
`GET` lista la colección; `POST` agrega un nuevo usuario a la colección.

Instancia: /api/v1/usuarios/{id}
`GET` consulta el usuario; `PUT` lo reemplaza; `DELETE` lo elimina.

Sub-recurso / Atributo: /api/v1/usuarios/{id}/estado
`PATCH` actualiza parcialmente el estado activo del usuario `{id}`.

Versionamiento de URLs y Contratos en APIs RESTful

1. URI PATH VERSIONING

En la Ruta (Estándar más Adoptado)
Incrusta el número de versión mayor directamente en el prefijo del URI:
`/api/v1/usuarios`
`/api/v2/usuarios`
Ventaja: Explícito, fácil de probar en navegador/Postman y de enrutar en API Gateways.

2. HEADER VERSIONING

Por Encabezado Personalizado
Mantiene la URI intacta y envía la versión solicitada en un header HTTP:
`GET /api/usuarios`
`X-API-Version: 1`
Ventaja: URIs limpias, aunque menos evidente al explorar o cachear por URL.

3. CONTENT NEGOTIATION

Por Media Type (Header Accept)
Versiona la representación usando el estándar MIME de negociación de contenido:
`Accept: application/vnd.icesi.v1+json`
Regla de Oro: Jamás rompas un contrato publicado; lanza `v2` mientras deprecias `v1`.

Universidad ICESI

Métodos Seguros

Idempotencia f(x)

Reintentos Seguros

Métodos HTTP REST: Seguridad e Idempotencia

1. Métodos Seguros (Safe Methods — RFC 9110)

Son operaciones de **solo lectura** que no alteran el estado del servidor (`GET` , `HEAD` , `OPTIONS`). Pueden ser pre-cargadas por navegadores o indexadas sin riesgo.

2. Idempotencia Matemática: f(f(x)) = f(x)

Una operación es **idempotente** si ejecutarla **N veces** seguidas con los mismos parámetros produce exactamente el **mismo estado en el servidor** que ejecutarla una sola vez (`GET` , `PUT` , `DELETE`).

3. ¿Por Qué POST NO es Idempotente?

Enviar 3 veces `POST /api/v1/cursos` crea 3 registros con IDs distintos. En cambio, enviar 3 veces `PUT /api/v1/usuarios/4` o `DELETE /api/v1/usuarios/4` deja el sistema en el mismo estado final.

Universidad ICESI

Matriz de Propiedades de los Métodos HTTP REST (RFC 9110)

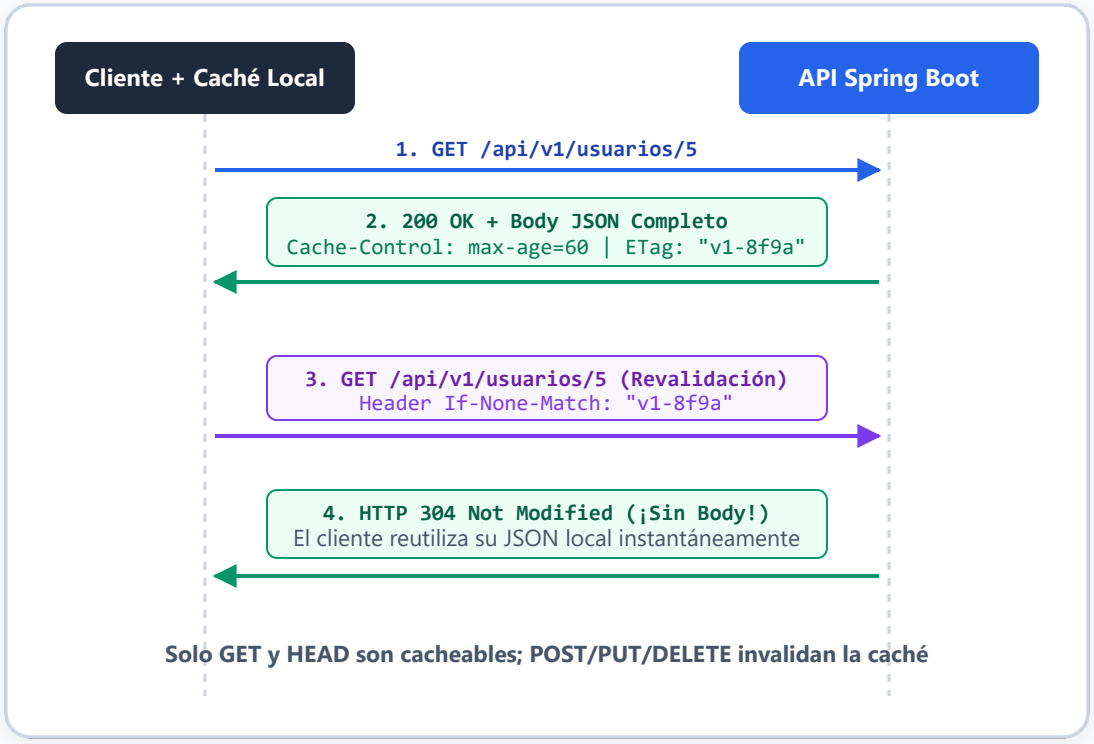
Método HTTP	Acción Semántica	¿Seguro?	¿Idempotente?	¿Cacheable?	Ejemplo de Endpoint RESTful
GET	Obtener recurso o colección	**SÍ**	**SÍ**	**SÍ (Por defecto)**	`GET /api/v1/usuarios?active=true`
POST	Crear nuevo recurso subordinado	**NO**	**NO (Crea N recursos)**	**NO**	`POST /api/v1/usuarios`
PUT	Reemplazar el recurso en su totalidad	**NO**	**SÍ (Sobrescribe igual)**	**NO**	`PUT /api/v1/usuarios/{id}`
PATCH	Actualizar parcialmente campos	**NO**	**Sí si asigna valor fijo**	**NO**	`PATCH /api/v1/usuarios/{id}/estado?active=false`
DELETE	Eliminar el recurso identificado	**NO**	**SÍ (Estado final borrado)**	**NO**	`DELETE /api/v1/usuarios/{id}`

Nota de Tolerancia a Fallos: En redes móviles inestables, los clientes HTTP reintentan automáticamente peticiones `GET`, `PUT` y `DELETE` ante un timeout porque su **idempotencia** garantiza que no corromperán ni duplicarán el estado.

Caché de Peticiones REST: Cache-Control, ETag y 304 Not Modified

Restricción REST Cacheable: Ahorro de Ancho de Banda y CPU

Las respuestas a peticiones GET pueden reutilizarse localmente o validarse mediante ETag condicional



CABECERAS ESTÁNDAR DE CACHÉ

- **Cache-Control: max-age=60**
  Durante 60 seg el cliente ni siquiera contacta al servidor (0 ms de red).
- **ETag / If-None-Match**
  Huella hash de la versión del recurso. Si no cambió, responde **304 Not Modified**.

SOPORTE NATIVO EN RESPONSEENTITY

```
return ResponseEntity.ok()
.cacheControl(
CacheControl.maxAge(60, SECONDS)
.cachePublic())
.eTag(String.valueOf(dto.hashCode()))
.body(dto);
```

Taxonomía de Códigos de Estado HTTP (HttpStatus)

2XX — ÉXITO

200 OK: Lectura (`GET`) o edición (`PUT/PATCH`) exitosa con JSON.

201 Created: Nuevo recurso creado (`POST`) + Header `Location` .

204 No Content: Borrado exitoso (`DELETE`) sin cuerpo.

3XX — CACHÉ & REDIR

304 Not Modified: El recurso en caché del cliente sigue vigente (valida `ETag` sin reenviar JSON).

302 Found: Redirección típica de MVC hacia `/login` (no deseada en clientes REST).

4XX — ERROR CLIENTE

400 Bad Request: JSON inválido.
401 Unauthorized: Sin token JWT válido.
403 Forbidden: Sin permisos suficientes.
404 Not Found: ID inexistente.
409 Conflict: Correo duplicado.

5XX — ERROR SERVIDOR

500 Internal Server Error: Falla no controlada en el backend.

503 Service Unavailable: Base de datos o microservicio caído.

Jamás devuelvas 200 OK cuando ocurrió un error.

Práctica Ciclo 1: Diagnóstico de Antipatrones en Controller.java

```java
// Código actual en controller/Controller.java
@RestController
public class Controller {

  //ANTIPATRÓN 1: Devuelve la @Entity JPA directa
  @GetMapping("/usuario")
  public List<Usuario> getUsuarios() {
    return usuarioService.obtenerTodosLosUsuarios();
  }

  //ANTIPATRÓN 2: ¡Muta la BD con un método GET!
  @GetMapping("/crear-usuario")
  public Usuario crearUsuario() {
    Usuario u = new Usuario();
    u.setCorreoInstitucional("example@icesi.edu.co");
    u.setPassword(passwordEncoder.encode("123456"));
    return usuarioService.crearUsuario(u);
  }
}
```

1. Fuga de Hashes BCrypt (Data Exposure)

Serializar `Usuario` expone `"password": "$2a$10$7s/F87..."` y la estructura interna de tablas `Rol` y `Permiso`.

2. Ruptura de Seguridad e Idempotencia

`GET /crear-usuario` usa un verbo en la URI y modifica el servidor mediante un método que debería ser seguro.

3. Sin Versionamiento ni ResponseEntity

Carece del prefijo `/api/v1/usuarios` y no distingue códigos de estado como `201 Created` o `404 Not Found`.

02. El Patrón DTO (Data Transfer Object)

Aislamiento del modelo de persistencia JPA, contratos asimétricos e implementación sobre Usuario.

¿Por Qué Jamás Devolver ni Recibir Entidades JPA (@Entity) en la API?

1. SEGURIDAD & PRIVACIDAD

Fuga de Datos y Mass Assignment
En lecturas (`GET`), devolver `Usuario` filtra el hash BCrypt `password` .

En escrituras (`POST/PUT`), recibir la entidad permite que un cliente malicioso inyecte `"id": 1` o se auto-asigne roles administrativos.

2. DESACOPLAMIENTO BD

Evolución Independiente
Si las tablas SQL cambian (ej. renombrar una columna o normalizar `Rol`), las aplicaciones móviles en producción se rompen.

El **DTO** blinda el contrato público frente a cambios internos del esquema JPA.

3. SERIALIZACIÓN & PESO

Ciclos Infinitos y Lazy Loading
Relaciones bidireccionales provocan `StackOverflowError` al serializar a JSON o disparan `LazyInitializationException` .

El DTO transporta únicamente los campos exactos que requiere la vista o cliente.

Asimetría de Contratos: UsuarioRequestDTO vs. UsuarioResponseDTO

Separación de Contratos de Entrada y Salida (Entidad Usuario)

La contraseña entra en el registro (RequestDTO), pero jamás sale en las consultas (ResponseDTO)

ENTRADA (POST / PUT)

UsuarioRequestDTO

```
+ nombre: String
+ apellido: String
+ correoInstitucional: String
+ password: String (Plano)
+ roles: Set<String>
```

(No permite enviar id ni active)

Evita ataques de **Mass Assignment** sobre el identificador primario.

toEntity →

MODELO JPA INTERNO

@Entity Usuario

```
- id: Long (@Id)
- nombre: String
- apellido: String
- correoInstitucional
- password: Hash BCrypt
- active: Boolean
- roles: Set<Rol> (EAGER)
```

Nunca sale de la capa Service

toDTO →

SALIDA SEGURA (JSON)

UsuarioResponseDTO

```
+ id: Long
+ nombre: String
+ apellido: String
+ correoInstitucional: String
+ active: Boolean
+ roles: Set<String> (Aplanado)
```

Sin campo password: Cero riesgo de filtrar hashes en respuestas HTTP.

- **Aplanamiento de Relaciones (Flattening):** Transformamos la entidad compleja `Set<Rol>` en una lista limpia de nombres `["ADMINISTRADOR"]`.

Práctica Ciclo 2: Codificación de los DTOs de Entrada y Salida

1. dto/UsuarioRequestDTO.java (Contrato de Entrada)

```java
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.util.Set;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UsuarioRequestDTO {

    private String nombre;
    private String apellido;
    private String correoInstitucional;
    private String password;
    // Ej: ["ADMINISTRADOR", "PROFESOR"]
    private Set<String> roles;
}
```

2. dto/UsuarioResponseDTO.java (Contrato de Salida Segura)

```java
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.util.Set;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UsuarioResponseDTO {

    private Long id;
    private String nombre;
    private String apellido;
    private String correoInstitucional;
    private Boolean active;
    // Aplanado de Set<Rol> a Set<String>
    private Set<String> roles;
}
```

Universidad ICESI

03. Java Streams API y Mapeo Estático con MapStruct

Procesamiento declarativo de colecciones con Streams y generación de mapeadores en tiempo de compilación.

Universidad ICESI

Java Streams API: Transformación y Filtrado Declarativo de Colecciones

Anatomía de un Pipeline Funcional con Java Streams

Reemplaza los bucles for imperativos por tuberías inmutables de evaluación perezosa (Lazy Evaluation)

1. FUENTE (SOURCE)

`coleccion.stream()`

Abre un flujo secuencial sobre `List<Usuario>` o `Set<Rol>` sin modificar la colección original.

`Stream<Rol>`

2. FILTRADO (INTERMEDIA)

`.filter(Predicate<T>)`

Evalúa una expresión lambda booleana y retiene solo los elementos que cumplen la condición.

`u -> u.getActive()`

3. TRANSFORMACIÓN

`.map(Function<T, R>)`

Convierte cada elemento de tipo `T` (ej. `Rol`) en un nuevo tipo `R` (`String`) con Method Reference `::`.

`.map(Rol::getNombre)`

4. TERMINAL (COLLECT)

`.collect() / .toList()`

Dispara la ejecución del pipeline y materializa el resultado en un `List` o en un `Set` sin duplicados.

`Collectors.toSet()`

COMPARATIVA: BUCLE IMPERATIVO VS. PIPELINE FUNCIONAL CON JAVA STREAMS

```
// Antes (7 líneas con mutación de estado externa):
Set<String> res = new HashSet<>(); for (Rol r : roles) { res.add(r.getNombre()); }

// Con Java Streams + Method Reference (Declarativo, conciso y thread-safe):
Set<String> res = roles.stream().map(Rol::getNombre).collect(Collectors.toSet());
```

Universidad ICESI

Java Streams Aplicados en la Capa de Servicio y Mapeo

1. Filtrado Dinámico en la Capa de Servicio

Usamos `.stream().filter(...).toList()` (introducido en Java 16+) para filtrar colecciones por criterios opcionales como `?active=true`:

```java
@Override
@Transactional(readOnly = true)
public List<Usuario> obtenerUsuariosPorEstado(Boolean active) {
    if (active == null) {
        return usuarioRepository.findAll();
    }
    return usuarioRepository.findAll()
            .stream()
            .filter(u -> active == u.isActive())
            .toList();
}
```

2. Transformación de Colecciones con .map()

Tanto en mapeadores manuales como en métodos personalizados de **MapStruct**, `.stream().map()` transforma colecciones enteras sin efectos secundarios:

```java
// Conversión Set<Rol> -> Set<String> en el Mapper
default Set<String> rolesToStrings(Set<Rol> roles) {
    if (roles == null) {
        return Collections.emptySet();
    }
    return roles.stream()
            .map(Rol::getNombre) // Method Reference
            .collect(Collectors.toSet());
}

// Conversión List<Usuario> -> List<UsuarioResponseDTO>
usuarios.stream().map(usuarioMapper::toResponseDTO).toList();
```

Estrategias de Conversión Entidad ↔ DTO: ¿Por Qué MapStruct?

Criterio	Manual (Setters)	Reflexión (ModelMapper)	MapStruct (JSR 269)
Fase de enlace	Compilación	Ejecución (Runtime)	**Compilación**
Rendimiento	Máximo (Nativo)	Bajo (Usa Reflexión)	**Máximo (Nativo)**
Detección de Error	En compilación	Falla en producción	**En mvn compile**
Mantenibilidad	Baja (Boilerplate)	Opaca al depurar	**Alta (Solo Interfaz)**

Sinergia Lombok + MapStruct: En `maven-compiler-plugin` declaramos `lombok-mapstruct-binding` para que Lombok genere los `getters/builders` antes de que MapStruct escriba la clase `Impl`.

Configuración en pom.xml (annotationProcessorPaths)

```
<annotationProcessorPaths>
  <!-- 1. Lombok genera getters, setters y @Builder -->
  <path>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
  </path>
  <!-- 2. Binding oficial entre Lombok y MapStruct -->
  <path>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok-mapstruct-binding</artifactId>
    <version>0.2.0</version>
  </path>
  <!-- 3. Procesador MapStruct (JSR 269) -->
  <path>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct-processor</artifactId>
    <version>1.5.5.Final</version>
  </path>
</annotationProcessorPaths>
```

Práctica Ciclo 3: Interfaz UsuarioMapper y Código Autogenerado

Lo que declaramos: mapper/UsuarioMapper.java

```java
@Mapper(componentModel = "spring")
public interface UsuarioMapper {

  @Mapping(target = "roles", source = "roles")
  UsuarioResponseDTO toResponseDTO(Usuario usuario);

  List<UsuarioResponseDTO> toResponseDTOList(List<Usuario> list);

  @Mapping(target = "id", ignore = true)
  @Mapping(target = "active", ignore = true)
  @Mapping(target = "roles", ignore = true)
  Usuario toEntity(UsuarioRequestDTO requestDTO);

  default Set<String> rolesToStrings(Set<Rol> roles) {
    if (roles == null) return Collections.emptySet();
    return roles.stream()
        .map(Rol::getNombre)
        .collect(Collectors.toSet());
  }
}
```

Lo que MapStruct genera en target/generated-sources

```java
@Component
public class UsuarioMapperImpl implements UsuarioMapper {

  @Override
  public UsuarioResponseDTO toResponseDTO(Usuario u) {
    if (u == null) return null;

    return UsuarioResponseDTO.builder()
        .id(u.getId())
        .nombre(u.getNombre())
        .apellido(u.getApellido())
        .correoInstitucional(u.getCorreoInstitucional())
        .active(u.getActive())
        .roles(rolesToStrings(u.getRoles()))
        .build();
  }
  // + toEntity() y toResponseDTOList() sin reflexión
}
```

Universidad ICESI

04. @RestController, Parámetros HTTP y ResponseEntity

Mapeo de rutas versionadas, @PathVariable, @RequestParam, @RequestBody y construcción de respuestas HTTP.

Universidad ICESI

Enrutamiento Semántico: @RestController y @RequestMapping

@RequestMapping("/api/v1/usuarios") en la Clase

Centraliza el prefijo del recurso y la versión de la API para evitar repetirlo en cada método del controlador.

Anotaciones Compuestas por Verbo HTTP

`@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping` y `@DeleteMapping` enlazan cada método con un verbo HTTP exacto.

Composición Jerárquica de la URI:

Prefijo de clase `/api/v1/usuarios` + sub-ruta del método `/{id}` ➔ `GET`

`/api/v1/usuarios/{id}`

```java
@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioRestController {

  private final IUsuarioService usuarioService;
  private final UsuarioMapper usuarioMapper;

  // GET /api/v1/usuarios
  @GetMapping
  public ResponseEntity<List<UsuarioResponseDTO>> listar() { ... }

  // GET /api/v1/usuarios/{id}
  @GetMapping("/{id}")
  public ResponseEntity<UsuarioResponseDTO> obtener(
  @PathVariable Long id) { ... }

  // POST /api/v1/usuarios
  @PostMapping
  public ResponseEntity<UsuarioResponseDTO> crear(
  @RequestBody UsuarioRequestDTO dto) { ... }
}
```

Las 4 Formas de Extraer Datos de una Petición HTTP en Spring

1. @PATHVARIABLE

Variables de Ruta
Extrae identificadores incrustados en el segmento jerárquico de la URI:
`GET /api/v1/usuarios/{id}`

Ideal para localizar un recurso único de forma obligatoria.

2. @REQUESTPARAM

Query Parameters
Extrae pares clave=valor tras el signo `?` en la URL:
`/api/v1/usuarios?active=true`

Soporta `required = false` y `defaultValue` para filtros y paginación.

3. @REQUESTBODY

Cuerpo JSON (Payload)
Deserializa automáticamente el cuerpo HTTP JSON hacia un DTO Java mediante Jackson:
`@RequestBody UsuarioRequestDTO`

Estándar en `POST` y `PUT`.

4. @REQUESTHEADER

Cabeceras HTTP
Captura metadatos enviados en los headers de la petición:
`X-Client-App: Mobile-Icesi`

Útil para trazabilidad, auditoría, versión o idioma.

Criterio de Diseño: ¿Cuándo Usar @PathVariable vs. @RequestParam?

Usa @PathVariable para Identidad Jerárquica

Cuando el dato es parte estructural de la URI que identifica **qué recurso específico** se está consultando o modificando:

```
// Identifica de forma única al usuario con ID = 10
@GetMapping("/{id}")
public ResponseEntity<UsuarioResponseDTO> obtenerPorId(
    @PathVariable("id") Long id) { ... }

// Sub-recurso: modifica el estado del usuario con ID = 10
@PatchMapping("/{id}/estado")
public ResponseEntity<UsuarioResponseDTO> cambiarEstado(
    @PathVariable Long id,
    @RequestParam Boolean active) { ... }
```

Usa @RequestParam para Filtrado Opcional

Cuando aplicas criterios de filtrado, búsqueda o paginación sobre una colección y el parámetro puede omitirse:

```
// Atiende tanto:
// GET /api/v1/usuarios        (active == null -> todos)
// GET /api/v1/usuarios?active=true (solo usuarios activos)
@GetMapping
public ResponseEntity<List<UsuarioResponseDTO>> listarUsuarios(
    @RequestParam(name = "active", required = false) Boolean active,
    @RequestHeader(name = "X-Client-App",
            defaultValue = "Postman") String clientApp) {
    ...
}
```

Universidad ICESI

Control Total de la Respuesta HTTP con ResponseEntity

`ResponseEntity<T>` representa la respuesta HTTP completa: permite configurar el **Status Code**, los **Headers** (como `Location` o `Cache-Control`) y el **Body tipado**.

ResponseEntity.ok(dto)
Retorna `200 OK` con el DTO serializado en el cuerpo JSON.

ResponseEntity.created(location).body(dto)
Retorna `201 Created` e incluye la cabecera `Location: .../api/v1/usuarios/7`.

ResponseEntity.noContent().build()
Retorna `204 No Content` sin cuerpo para eliminaciones `DELETE`.

```
// 1. Construcción de URI Location dinámica en POST (201 Created)
URI location = ServletUriComponentsBuilder
    .fromCurrentRequest()
    .path("/{id}")
    .buildAndExpand(guardado.getId())
    .toUri();

return ResponseEntity
    .created(location)
    .body(usuarioMapper.toResponseDTO(guardado));

// 2. Respuesta con Caché HTTP en GET (200 OK + Cache-Control)
return ResponseEntity.ok()
    .cacheControl(CacheControl.maxAge(30, TimeUnit.SECONDS))
    .header("X-Processed-For-Client", clientApp)
    .body(response);

// 3. Respuesta de error semántico (404 Not Found / 409 Conflict)
return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
```

Universidad ICESI

Práctica Ciclo 4 (Parte A): GET y POST en UsuarioRestController

1. GET Colección (con Filtro) y GET por ID

```java
@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioRestController {

  @GetMapping
  @PreAuthorize("hasAuthority('USER_READ')")
  public ResponseEntity<List<UsuarioResponseDTO>> listar(
  @RequestParam(required = false) Boolean active,
  @RequestHeader(name = "X-Client-App", defaultValue = "Web") String app) {
    List<Usuario> list = usuarioService.obtenerUsuariosPorEstado(active);
    return ResponseEntity.ok()
        .header("X-Processed-For-Client", app)
        .body(usuarioMapper.toResponseDTOList(list));
  }

  @GetMapping("/{id}")
  @PreAuthorize("hasAuthority('USER_READ')")
  public ResponseEntity<UsuarioResponseDTO> obtener(@PathVariable Long id) {
    try {
      Usuario u = usuarioService.obtenerUsuarioPorId(id);
      return ResponseEntity.ok(usuarioMapper.toResponseDTO(u));
    } catch (IllegalArgumentException e) {
      return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
  }
```

2. POST Creación (201 Created + 409 Conflict)

```java
@PostMapping
@PreAuthorize("hasAuthority('USER_CREATE')")
public ResponseEntity<UsuarioResponseDTO> crear(
@RequestBody UsuarioRequestDTO requestDTO) {

  if (usuarioService.existeCorreo(
          requestDTO.getCorreoInstitucional())) {
    return ResponseEntity
        .status(HttpStatus.CONFLICT).build();
  }

  Usuario entidad = usuarioMapper.toEntity(requestDTO);
  Usuario guardado = usuarioService.crearUsuario(
      entidad, requestDTO.getRoles()
  );

  URI location = ServletUriComponentsBuilder
      .fromCurrentRequest()
      .path("/{id}")
      .buildAndExpand(guardado.getId())
      .toUri();

  return ResponseEntity
      .created(location)
      .body(usuarioMapper.toResponseDTO(guardado));
}
```

Práctica Ciclo 4 (Parte B): PUT, PATCH y DELETE en UsuarioRestController

3. PUT (Reemplazo) y PATCH (Estado Parcial)

```
@PutMapping("/{id}")
@PreAuthorize("hasAuthority('USER_UPDATE')")
public ResponseEntity<UsuarioResponseDTO> actualizar(
@PathVariable Long id, @RequestBody UsuarioRequestDTO req) {
  try {
    Usuario datos = usuarioMapper.toEntity(req);
    Usuario act = usuarioService.actualizarUsuario(id, datos, req.getRoles());
    return ResponseEntity.ok(usuarioMapper.toResponseDTO(act));
  } catch (IllegalArgumentException e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
  }
}
@PatchMapping("/{id}/estado")
@PreAuthorize("hasAuthority('USER_UPDATE')")
public ResponseEntity<UsuarioResponseDTO> cambiarEstado(
@PathVariable Long id, @RequestParam Boolean active) {
  try {
    Usuario u = usuarioService.cambiarEstadoActivo(id, active);
    return ResponseEntity.ok(usuarioMapper.toResponseDTO(u));
  } catch (IllegalArgumentException e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
  }
}
```

4. DELETE Semántico (204 No Content)

```
@DeleteMapping("/{id}")
@PreAuthorize("hasAuthority('USER_DELETE')")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
  try {
    usuarioService.eliminarUsuario(id);
    return ResponseEntity.noContent().build();
  } catch (IllegalArgumentException e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
  }
}
```

Puente hacia la Sesión 2: Al invocar `/api/v1/usuarios` en Postman sin sesión, un filtro `SecurityFilterChain` MVC tradicional redirige al HTML `/login`. En la Sesión 2 configuraremos una cadena Stateless con **JWT**.

Universidad ICESI

Fin de la Sesión 1

Próxima Sesión: Filtros, SecurityFilterChain Dual, OncePerRequestFilter y Autenticación Stateless con JWT